PRAKTIKUM 10
STRING

10.1 Tujuan

1. Mahasiswa dapat memahami penggunaan tipe data *string*.
2. Mahasiswa mampu menyusun program dengan memanfaatkan operasi *string*.

10.2 Landasan Teori

10.2.1 Pengantar *String*

Dalam mengolah, memanipulasi dan menampung data teks, dapat menggunakan bentuk data yang disebut dengan *String*. Sebagai contoh fungsi *string* misalnya digunakan untuk menyimpan sebuah kalimat dan lain sebagainya.

Penggunaan *String* dalam Bahasa pemrograman C atau C++, bukan untuk mendeklarasikan suatu tipe data. *String* merupakan bentuk data yang berdiri sendiri yang terdiri dari kumpulan nilai-nilai karakter secara urut menjadi bentuk *array* satu dimensi (belajarcpp.com (a), 2018). Sehingga untuk memanggil *string* akan selalu dalam bentuk karakter ASCII. Diakhir sebuah program akan ditandai dengan adanya karakter *NULL* atau '\0' yang merupakan nilai karakter *null* konstanta. Kemudian pernyataan tersebut disimpan dalam bentuk karakter variabel seperti dibawah ini:

```
char ch1, ch2, ch3;
ch1 = '\0';
ch2 = 0;
ch3 = NULL;
```

Sintaks 10. 1 *Penyimpanan Karakter Akhir String dalam Variabel Array*

Array merupakan sebuah memori yang berurutan untuk menyimpan nilai tipe data yang sama, akan tetapi pada *string* berupa *array* yang selalu diakhiri dengan karakter *null*. *String* dalam bahasa C++ akan muncul dalam program berupa salah satu dari tiga bentuk dibawah ini:

a. "*Hard-coded*'' *string* literals
b. *Programmer* mendefinisikan *array* dari karakter
c. *Pointers* ke karakter

Tanpa memperhatikan dari ketiga bentuk *string* yang muncul dalam program tersebut, *string* selalu mempunyai ciri *array* yang diakhiri oleh karakter *null* dan direpresentasikan dalam program dalam bentuk *pointer* pada karakter pertama dalam sebuah *array*. Dengan kata lain tipe *string* pada bahasa C++ adalah:

```
char *
```

Sintaks 10. 2 *Sintaks String dalam C++*

Sintaks 10.2 adalah tipe *char string* menggunakan sebuah *pointer*.

10.2.2 Konstanta *String*

Penulisan *String* Secara harafiah sebenarnya dapat disebut juga dengan konstanta *string* yang dapat ditulis secara langsung dalam program secara berurutan dengan menambah tanda ketik dua. Perhatikan *string* dibawah ini:

"siapakah nama anda?"

"gatotkaca"

Kedua penulisan tersebut diatas merupakan cara penulisan *string*. Perhatikan contoh sintaks 10.3 berikut:

```
#include <iostream>

using namespace std;

int main()
{
     cout << "gatotkaca";
     return 0;
}
```

Sintaks 10. 3 *Contoh Program Sederhana untuk Menampilkan String*

Tampilan program ketika dijalankan:

```
gatotkaca
Process returned 0 (0x0)   execution time : 0.273 s
Press any key to continue.
```

Gambar 10. 1 *Tampilan program ketika Sintaks 10.3 dijalankan*

Pada sintaks 10.3 dapat diketahui bahwa *compiler* menangani *string* dalam bentuk *string* literal. Ketika *compiler* menemui sebuah *string* "gatotkaca", maka hal tersebut akan dialokasikan dalam sebuah *array* dengan panjang sepuluh karakter dalam bentuk data dalam program. *Compiler* kemudian akan menyimpan sembilan karakter dalam bentuk *string* "gatotkaca". Dalam penyimpanan pertama dimasukan hingga karakter kesembilan ke dalam *array* dan di akhiri dengan karakter *null* pada penyimpanan terakhir. Langkah terakhir *compiler* akan melewatkan nilai tipe *char* * dan alamat *Array* dari karakter pertama menjadi obyek cout.

g	a	t	o	t	k	a	c	a	\0

Gambar 10. 2 *Penyimpanan String dalam Bentuk Array*

Pada gambar 10.2 setiap karakter pada tiap elemen atau indeks *array* akan menggunakan memori sebesar 1 *byte*. Secara otomatis *byte* terakhir akan berisikan karakter *NULL* yang menandakan bahwa *string* tersebut sudah berakhir. Program di bawah ini mengilustrasikan sebuah *string* yang diperlakukan oleh *compiler* sebagai sebuah nilai dengan type data **char*** . Hal tersebut merupakan sebuah operasi *pointer* pada sebuah karakter.

10.2.3 Variabel *String*

Untuk memiliki suatu karakter *string*, harus mendefinisikan sebuah *array* untuk menampung karakter *string* tersebut. Dalam melakukan hal ini, yang perlu dipastikan adalah untuk mengalokasikan satu tambahan isi *array* untuk mengakhiri karakter *null*. Sebagai contoh, jika *string* yang panjangnya 19 karakter, maka perlu menyediakan sebuah a*rray* dengan Panjang 20 karakter, seperti contoh penulisan dibawah ini:

```
char company[20];
```

Sintaks 10. 4 *Sintaks Pendeklarasian Variabel String*

Instruksi pada sintaks 10.4 merupakan instruksi yang digunakan untuk mendeklarasikan variabel *string* dengan panjang maksimal 20 karakter (termasuk karakter *NULL*).

10.2.4 Library String

Berikut beberapa *function* yang disediakan C++ untuk operasi *string*:

1. strcpy

   Digunakan untuk meng-*copy string*.

```
strcpy(kata2, kata1); // menyalin isi dari kata1 ke
kata2
```

Sintaks 10. 5 *Contoh Penggunaan Sintaks strcpy*

2. strlen

   Digunakan untuk mengetahui panjang karakter *string*.

```
strlen(string);
```

Sintaks 10. 6 *Contoh Penggunaan Sintaks strlen*

   Nilai keluarannya adalah bilangan bulat dimana bilangan tersebut menyatakan panjang *string*.

3. strcat

   Digunakan untuk menggabungkan *string*.

```
strcat(string1, string2)
```

Sintaks 10. 7 *Contoh Penggunaan Sintaks strcat*

   Nilai keluarannya adalah gabungan antara *string*1 dan *string*2 dengan menambahkan *string*2 ke *string*1.

4. strupr

   Digunakan untuk mengubah semua huruf menjadi huruf kapital.

```
char string1[30] = "aBcDefgHIJKLmno";
strupr(string1); //string1 menjadi "ABCDEFGHIJKLMNO"
```

Sintaks 10. 8 *Contoh Penggunaan Sintaks strupr*

   Mengubah huruf kecil pada tiap elemen *string* ke huruf kapital.

5. strlwr

Digunakan untuk mengubah semua huruf menjadi huruf kecil.

```
strlwr(string
```

Sintaks 10. 9 *Contoh Penggunaan Sintaks strlwr*

Function ini adalah kebalikan dari *function* `strupr()`.

6. substr

Digunakan untuk mencari sub*string*.

Misalkan terdapat *string* dengan "JAKARTA KOTA METROPOLITAN".

Apakah *string* "METRO" terdapat pada *string* tersebut?

Hal tersebut dapat diketahui jika menggunakan *function* strstr().

Sintaks:

```
strstr(string1, string2);
```

Sintaks 10. 10 *Format Penulisan Sintaks strstr*

Function tersebut akan mengembalikan sebuah nilai. Apabila *string* 2 merupakan bagian atau sub*string* dari *string* 1 maka akan mengembalikan nilai 1, jika tidak maka nilai 0 yang akan menjadi hasil dari pengembalian nilai.

Contoh:

```
if (strstr("JAKARTA KOTA METROPOLITAN", "METRO") == 1)
      cout << "Merupakan substring";
else
      cout << "Bukan merupakan substring";
```

Sintaks 10. 11 *Contoh Penggunaan Sintaks strstr dalam Program*

7. strrev

Digunakan untuk membalik *string*.

Contoh, bagaimana cara membalik "C++" supaya menjadi "++C"? Berikut ini perintah dalam C++.

```
char kata[10] = "C++";
strrev(kata);
cout << kata;
```

Sintaks 10. 12 *Contoh Penggunaan Sintaks strrev*

10.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 11
PENGURUTAN *ARRAY*

11.1 Tujuan

1. Mahasiswa mampu menyusun program pengurutan menggunakan *Array*.

11.2 Landasan Teori

11.2.1 Pengantar Pengurutan *Array*

Sorting atau pengurutan adalah sebuah proses pengurutan data yang semula tersusun secara tidak beraturan sehingga data itu tersusun secara teratur atau terurut menurut suatu aturan tertentu (Admin Sarjana (a), 2019). Proses pengurutan dapat dilakukan melalui dua macam urutan yaitu pengurutan dari kecil ke besar atau urut menaik (*ascending*) dan pengurutan dari besar ke kecil atau urut menurun (*descending*). Salah satu tujuan utama proses pengurutan adalah agar data dapat lebih mudah dilihat dan diolah. Contoh: data bilangan 7, 4, 8 dan 6 dapat diurutkan naik menjadi 4, 6, 7, 8 atau diurutkan turun menjadi 8, 7, 6, 4.

Keuntungan dari data yang sudah terurut antara lain:

a. Mudah dalam pencarian data (misalnya dalam kamus bahasa atau buku telepon), mudah untuk dibetulkan, digabungkan, disisipi atau dihapus. Dalam kondisi terurut, kita dapat dengan mudah melakukan pengecekan apakah ada data yang hilang atau kurang.
b. Melakukan kompilasi program komputer jika tabel-tabel simbol harus dibentuk.
c. Mempercepat proses dalam pencarian suatu data yang harus dilakukan secara berulang.

Sorting dapat dilakukan menggunakan beberapa metode, diantaranya:

a. *Bubble sort*
b. *Merger sort*
c. *Selection sort*
d. *Insertion sort*
e. *Quick sort*

f. *Shell sort*

g. *Heap sort*

Namun pada bab ini, kita hanya akan mempelajari lebih dalam pada beberapa metode saja yang lebih umum digunakan.

11.2.2 *Bubble Sort*

Bubble Short atau metode gelembung adalah metode yang digunakan dalam pengurutan data dalam suatu a*rray* dengan membandingkan nilai dari tiap-tiap elemen *array* (Admin Sarjana (a), 2019). Setelah dibandingkan, dapat dilakukan penukaran nilai dari elemen *array* jika diperlukan. Metode ini biasa disebut *Exchange Sort* atau metode penukaran.

Pengurutan *Ascending*: Penukaran dari elemen saat ini dengan elemen setelahnya apabila nilai elemen saat ini diketahui lebih besar.

Pengurutan *Descending*: Penukaran dari elemen saat ini dengan elemen setelahnya apabila nilai elemen saat ini diketahui lebih kecil.

Algoritma ini seolah-olah akan menggeser satu per satu dari tiap elemen dari kiri ke kanan atau sebaliknya, tergantung pada jenis pengurutannya. Ketika satu proses telah terselesaikan, maka metode gelembung akan terus mengulangi proses. Kapan proses tersebut berhenti? *Bubble sort* akan berhenti jika seluruh elemen *array* telah dilakukan pengecekan dan tidak memungkinkan adanya pertukaran lagi yang bisa dilakukan, serta telah mencapai pengurutan yang telah diinginkan. Metode ini memungkinkan kita untuk memahaminya dengan mudah, akan tetapi metode *Bubble Sort* ini merupakan metode yang paling tidak efisien dalam penggunaannya.

Bubble Sort akan melalukan beberapa tahapan sebagai berikut:

a. Pengecekan dapat dilakukan dari data yang paling depan atau awal maupun paling akhir. Jika pengecekkan dilakukan dari data yang paling awal, maka data paling awal akan dilakukan pengecekan dengan data yang ada setelahnya. Jika ternyata data yang terdapat di awal tersebut lebih kecil, maka kedua data tersebut akan ditukar. Proses pengecekan tersebut dilakukan hingga pada data yang paling akhir.

b. Langkah selanjutnya kembali dilakukan pengecekan pada data paling awal dan dibandingkan lagi dengan data yang ada setelahnya. Jika ditemukan data yang berada di awal tersebut lebih kecil, maka kedua data tersebut akan ditukar dan proses pengecekan dilakukan kembali, namun pengecekan tidak akan dilakukan hingga pada data yang terakhir. Karena data paling akhir merupakan data yang paling kecil. Langkah ini akan dilakukan berulang secara terus menerus sesuai jumlah data yang telah dimasukkan. .

Berikut prosedur *Bubble Sort*:

```
void BubbleSort() {
   int i, j;

   for(i=1; i<Max-1; i++)
      for(j=Max-1; j>=i; j--)
         if(Data[j-1] > Data[j])
            Tukar(&Data[j-1], &Data[j]);
}
```

Sintaks 11. 1 *Contoh Program Menggunakan Bubble Sort*

11.2.3 *Selection Sort*

Selection Sort atau metode seleksi adalah proses pengurutan data pada *array* dengan cara mencari data yang paling kecil kemudian menukarnya dengan data yang digunakan sebagai acuan atau sering disebut pivot (Admin Sarjana (a), 2019).

Proses pengurutan dengan metode *selection sort* dilakukan dengan cara sebagai berikut:

a. Langkah pertama mencari data paling kecil dari data paling awal atau data elemen pertama sampai data terakhir atau elemen paling akhir. Kemudian data terkecil tersebut akan ditukar dengan data yang terdapat pada elemen pertama *array*. Dengan demikian, data pada elemen pertama mempunyai nilai terkecil jika dibandingkan dengan data pada elemen yang lain.

b. Langkah kedua, mencari data paling kecil kembali dimulai dari data elemen kedua sampai elemen terakhir. Data terkecil yang diperoleh kemudian ditukar dengan data elemen kedua dan begitu seterusnya hingga semua data tiap elemen pada *array* dalam keadaan terurutkan.

Berikut prosedur *Selection Sort:*

```
void SelectionSort() {
   int i, j, k;

   for(i=0; i<Max-1;i++){
      k = i;

      for (j=i+1; j<Max; j++)
         if(Data[k] > Data[j])
            k = j;
         Tukar(&Data[i], &Data[k]);
   }
}
```

Sintaks 11. 2 *Contoh Program Menggunakan Selection Sort*

11.2.4 *Insertion Sort*

Proses mengurutkan data dengan cara membandingkan data ke-Z (dimana Z dimulai dari data elemen ke-2 sampai dengan pada data elemen terakhir sebuah *array*) dengan data setelahnya dapat disebut dengan metode pengurutan *Insertion Sort* (Admin Sarjana (a), 2019). Jika ditemukan adanya data yang lebih kecil dari data sebelumnya, maka data tersebut akan disisipkan ke depan sesuai dengan posisi yang sesuai.

Pada metode ini juga terdapat kelemahan dalam penggunaannya yaitu terlalu banyak proses pergeseran yang dilakukan untuk mencari posisi yang sesuai. Banyaknya proses pergeseran meningkat secara kuadratik sesuai dengan jumlah Z yang besar. Metode kurang bagus jika dilakukan untuk jumlah data yang besar.

Berikut prosedur *Insertion Sort*:

```
void insertion_sort () {
  int temp;
  for(int i=1; i<n; i++) {
    temp = data[i];
    j = i-1;
    while (data[j]>temp && j>=0) {
      data[j+1] = data[j];
      j--;
    }
    data[j+1] = temp;
  }
}
```

Sintaks 11. 3 *Contoh Program Menggunakan Insertion Sort*

11.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 12
PENCARIAN *ARRAY*

12.1 Tujuan

1. Mahasiswa mampu menyusun program pencarian menggunakan *Array*.

12.2 Landasan Teori

12.2.1 Pengantar Pencarian *Array*

Pencarian *Array* (*Searching)* adalah proses untuk menemukan suatu data atau nilai tertentu pada sebuah *array* baik pada *array* satu dimensi maupun dua dimensi.

Berikut jenis-jenis pencarian:

a) Pencarian Beruntun (Sekuensial *Search*)

b) Pencarian Bagi dua (*Binary Search*)

12.2.2 Pencarian Beruntun (Sekuensial *Search*)

Pencarian *Array* secara beruntun dengan cara menelusuri satu persatu elemen dari data yang ada pada *array*. Tiap elemen akan dilakukan pengecekan dan dibandingkan apakah data pada elemen tersebut sama atau sesuai dengan data yang dicari. Jika data pada elemen yang sedang di cek sudah sesuai, maka proses pencarian data akan dihentikan. Jika tidak sesuai maka proses pencarian akan terus dilanjutkan hingga data yang dicari ditemukan. Pencarian a*rray* dengan metode ini dapat dilakukan pada data *array* yang sudah terurut maupun belum terurut (Admin Sarjana (b), 2019).

Contoh pencarian data pada a*rray* yang sudah terurut:

```
#include <iostream>
#include <stdio.h>
#include <conio.h>

using namespace std;

int main()
{
    int larik[9]={1,12,3,4,10,6,7,11,9}, i,n, x, posisi;

    cout<<"Data = ";
    for(int i = 0; i<9; i++){
        cout << larik[i] << " ";
    }
    cout<<endl<<"data yang ingin dicari ? ";cin>>x;

    i=0;
    posisi=0;
    while(i<8 && larik[i]!=x)
    {
     i++;
    }

    if (larik[i]!=x)
     cout<<"maaf data yang dicari tidak ada";
    else if (larik[i]==x)
    {
     posisi=i+1;
     cout<<"pada posisi ke "<<posisi;
    }

    getch();
}
```

Sintaks 12. 1 *Contoh Program Pencarian Data pada Array yang Sudah Terurut*

Tampilan program ketika dijalankan:

```
Data = 1 12 3 4 10 6 7 11 9
data yang ingin dicari ? 10
pada posisi ke 5
```

Gambar 12. 1 *Tampilan program ketika Sintaks 12.1 dijalankan*

Contoh pencarian data pada *Array* yang belum terurut:

```
#include <stdio.h>
#include <conio.h>

using namespace std;

int main (void) {
    int i, n, dt, posisi, ketemu = 0;
    int data[100];

    printf ("berapa banyak data di dalam Array: ");
    scanf ("%i", &n);

    for (i = 0; i < n; i++) {
        printf("masukkan data ke-%i = ", (i+1));
        scanf ("%i", &(*(data+i)));
    }

    printf("data yang dicari: ");
    scanf ("%i", &dt);

    ketemu = 0;
    for (i = 0; i < n; i++) {
        if (*(data+i) == dt) {
            ketemu = 1;
            posisi = i;
            printf  ("Data:  \"%i\"  ditemukan  di  posisi=
    %i\n",
            *(data+i), (posisi+1));
        }
    }

    if (ketemu == 0) printf("data tidak ditemukan!\n");
    getch();
}
```

Sintaks 12. 2 *Contoh Program Pencarian Data pada Array yang Belum Terurut*

Tampilan program ketika dijalankan:

```
berapa banyak data di dalam Array: 3
masukkan data ke-1 = 2
masukkan data ke-2 = 5
masukkan data ke-3 = 4
data yang dicari: 5
Data: "5" ditemukan di posisi= 2
```

Gambar 12. 2 *Tampilan program ketika Sintaks 12.2 dijalankan*

12.2.3 Pencarian Bagi Dua (*Binary Search*)

Syarat: Data harus terurut.

Proses pencarian dengan cara ini yaitu dengan membagi dua bagian dari *array* yang ada (Admin Sarjana (b), 2019). Sebelum itu, *array* harus dalam keadaan urut sehingga memudahkan proses pencarian. Seperti halnya kita dalam mencari sebuah kata dalam sebuah kamus, kita tidak perlu membuka atau melakukan pengecekan dari awal halaman. Kita dapat langsung membagi halaman dari kamus tersebut sesuai dengan data yang akan dicari.

Untuk melakukan pencarian menggunakan metode ini akan ada beberapa proses yang terjadi, yaitu:

1) Membaca *array* data.
2) Mengurutkan data pada *array* terlebih dahulu.
3) Menentukan data yang akan dicari.
4) Menentukan elemen tengah *array*. Letak posisi tengah dapat ditentukan dengan `Tengah = (n div 2) + 1.`
5) Jika nilai pada elemen tengah sebuah *array* sama dengan data yang akan dicari, maka pencarian suatu data dinyatakan selesai.
6) Jika nilai pada elemen tengah sebuah *array* tidak sama dengan data yang akan dicari, maka:
   a. Akan dilakukan pencarian dari elemen pertama hingga elemen tengah sebuah *array* jika nilai pada elemen yang berada di tengah lebih besar dari pada data yang dicari.
   b. Akan dilakukan pencarian dari elemen tengah hingga elemen terakhir sebuah *array* jika nilai pada elemen yang berada di tengah lebih kecil dari pada data yang dicari.

Contoh program:

```
#include <iostream>
#include <conio.h>

using namespace std;

int main(){
    int tar, i, j, temp, n;
    int data[100];
    cout << "Banyak data = ";
    cin >> n;

    for(i=0; i<n; i++){
        cout << "Masukkan data ke-" << i+1 << " = ";
        cin >> data[i];
    }

    for(int i=1; i<n; i++) {
        temp = data[i];
        j = i-1;
        while (data[j]>temp && j>=0) {
            data[j+1] = data[j];
            j--;
        }
        data[j+1] = temp;
    }
    cout << "Setelah pengurutan: " << endl;

    for(i=0; i<n; i++)
        cout << data[i] << "  ";

    cout<< endl << "masukan data yang dicari: ";
    cin>>tar;
    int awal=0, akhir=10, tengah;

    while (awal <= akhir){
        tengah = (awal + akhir)/2;
        if (tar > data[tengah] )
                awal = tengah + 1;
        else if (tar < data[tengah])  // descending ubah
tanda < menjadi >
            akhir= tengah - 1;
        else awal = akhir +1;
    }
   if (tar == data[tengah])
        cout<<"Data ditemukan, Ke- "<<tengah+1<<endl;
    else
        cout<<"target tidak ditemukan "<<endl;
}
```

Sintaks 12. 3 *Contoh Program Pencarian Data Menggunakan Binary Search*

Tampilan program ketika dijalankan:

```
Banyak data = 4
Masukkan data ke-1 = 3
Masukkan data ke-2 = 0
Masukkan data ke-3 = 9
Masukkan data ke-4 = 7
Setelah pengurutan:
0  3  7  9
masukan data yang dicari: 7
Data ditemukan, Ke- 3

Process returned 0 (0x0)   execution time : 18.255 s
Press any key to continue.
```

Gambar 12. 3 *Tampilan program ketika Sintaks 12.3 dijalankan*

12.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 13
STRUKTUR (*STRUCT*)

13.1 Tujuan

1. Mahasiswa mampu memahami konsep *struct*.
2. Mahasiswa mampu menyusun program menggunakan *struct*.

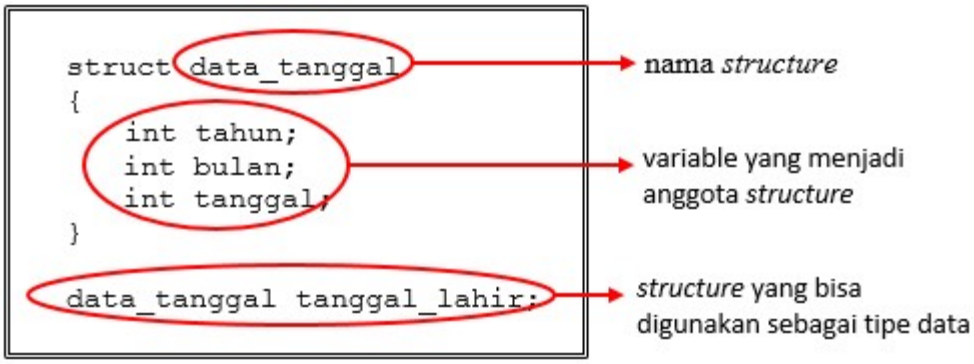
13.2 Landasan Teori

13.2.1 Pengertian Struktur (*Struct*)

Variabel-variabel atau beberapa data yang mempunyai tipe data yang berbeda dikelompokkan menjadi satu dalam nama yang sama (Prastio, 2015). Biasanya struktur digunakan dalam pengelompokkan informasi-informasi yang berhubungan satu sama lain, atau sering disebut *record.*

13.2.2 Deklarasi Struktur (*Struct*)

Untuk mendeklarasikan sebuah struktur dengan menggunakan kata kunci *struct* yang nantinya berfungsi layaknya sebuah tipe data:



Gambar 13. 1 *Deklarasi Sebuah Struktur (Struct)*

Dapat dilihat pada contoh potongan program di atas, terdapat sebuah struktur yang mempunyai nama data_tanggal, struktur tersebut tersusun dari 3 bagian yaitu tahun, bulan dan tanggal. Struktur ini nantinya dapat digunakan sebagai tipe data dari variabel yang digunakan atau dapat disebut tipe data abstrak, disini variabel yang digunakan adalah data_tanggal. Berikut contoh programnya.

Contoh program:

```
#include<iostream>
#include<conio.h>

using namespace std;

int main()
{
    struct data_tanggal
    {
        int tanggal;
        int bulan;
        int tahun;
    };

    data_tanggal tanggal_lahir;
    tanggal_lahir.tanggal = 19;
    tanggal_lahir.bulan = 9;
    tanggal_lahir.tahun = 2019;

    cout    <<    tanggal_lahir.tanggal    <<    "/"    <<
tanggal_lahir.bulan << "/" << tanggal_lahir.tahun;
}
```

Sintaks 13. 1 *Contoh Program Menggunakan Struct*

Tampilan program ketika dijalankan:

```
19/9/2019
Process returned 0 (0x0)   execution time : 0.239 s
Press any key to continue.
```

Gambar 13. 2 *Tampilan program ketika Sintaks 13.1 dijalankan*

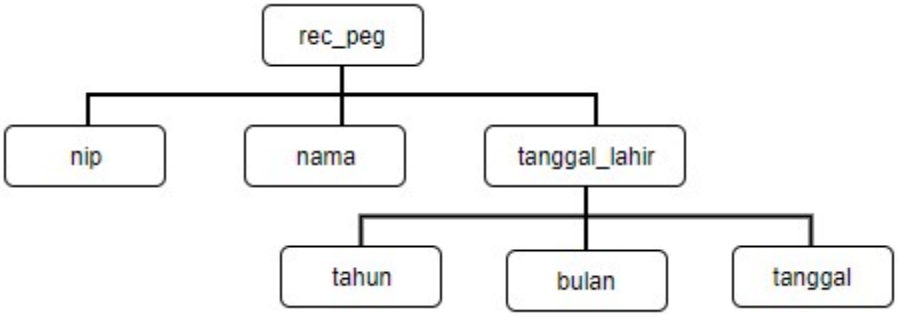
13.2.3 Struktur dalam Struktur

Dalam sebuah struktur di dalamnya juga dapat mengandung satu atau lebih struktur lain. Sebagai contoh dapat dilihat seperti di bawah ini:

```
struct data_pegawai {
  int nip;
  char nama[30];
  data_tanggal tanggal_lahir;
}rec_peg;
```

Sintaks 13. 2 *Format Penulisan Struct dalam Struct*

Dapat dilihat pada contoh sintaks 13.2 di atas, terdapat sebuah struktur yang di deklarasikan dengan nama data_pegawai dimana pendeklarasian tersebut juga mendeklarasi variabel struktur yang diberi nama rec_peg.



Gambar 13. 3 *Bagan Struktur di dalam Struktur*

13.2.4 Mengakses Anggota Struktur

Untuk mengakses bagian dari anggota struktur dengan cara sebagai berikut:

```
variable_struktur.nama_anggota
```

Sintaks 13. 3 *Sintaks untuk Mengakses Anggota Struct*

Contoh:

```
rec_peg.nip = 108533414505;
rec_peg.nama = "dewi";
rec_peg.tanggal_lahir.tahun = 2009;
```

Sintaks 13. 4 *Contoh Pemanggilan Anggota Struct*

13.2.5 Perbandingan 2 Buah Struktur

Struktur satu juga dapat dibandingkan dengan struktur lain dengan membandingkan dari masing-masing bagian pada kedua struktur tersebut.

Contoh:

```
if ((tgl1.tanggal == tgl2.tanggal) && (tgl1.bulan ==
tgl2.bulan) && (tgl1.tahun == tgl2.tahun))
     cout << "Isi strukturnya sama";
else
     cout << "Isi struktur tak sama";
```

Sintaks 13. 5 *Contoh Penggunaan Struct untuk Perbandingan Nilai*

13.2.6 Struktur dalam Fungsi

Struktur juga bisa digunakan sebagai sebuah parameter dalam sebuah *function*.

Contoh 3:

```
#include<iostream>
#include<conio.h>

using namespace std;

struct data
{
    int x;
    int y;
};

void tampilkan(data nilai);

int main()
{
    data nilaiku;
    nilaiku.x = 10;
    nilaiku.y = 16;
    tampilkan(nilaiku);
}

void tampilkan(data nilai)
{
    cout << "Nilai x = " << nilai.x << endl;
    cout << "Nilai y = " << nilai.y << endl;
}
```

Sintaks 13. 6 *Contoh Program Menggunakan Struct*

Tampilan program ketika dijalankan:

```
Nilai x = 10
Nilai y = 16

Process returned 0 (0x0)   execution time : 0.250 s
Press any key to continue.
```

Gambar 13. 4 *Tampilan program ketika Sintaks 13.6 dijalankan*

13.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.